



物件導向程式設計

撲克牌21點

系級：資工IA

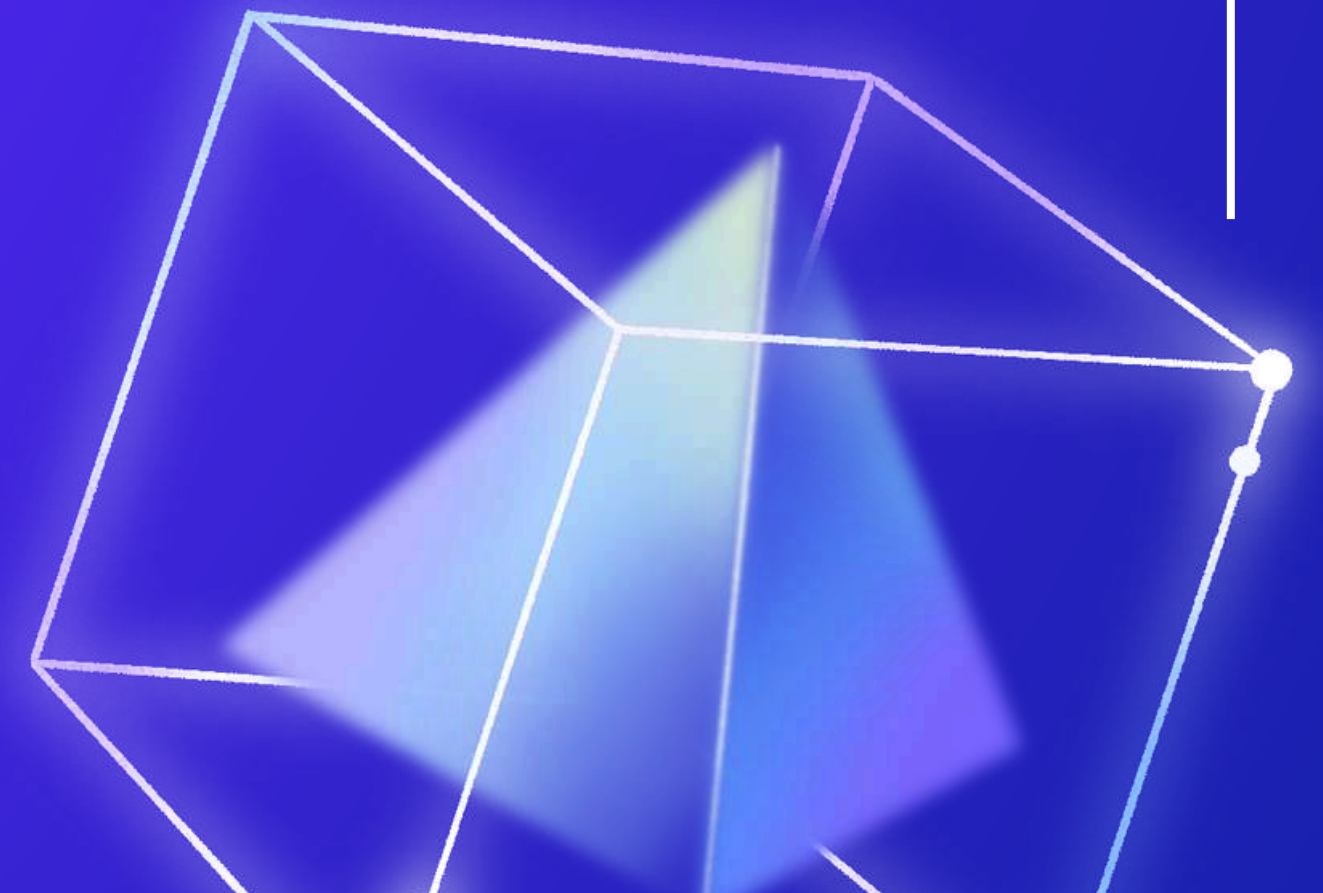
組長：黃泓霖

組員：王煒竣、李厚侑



目錄:

1. 專題內容說明	03
• 系統之整體介紹	04
• 系統流程圖	05
• 系統功能描述	06-11
• 程式架構介紹(主要類別、物件、封裝、繼承、多型、以及核心邏輯)	
• 系統使用方式	12
• 系統運行畫面截圖	13
2. UML圖	14
3. 程式運作流程圖 (Flowchart)	15
4. GitHub 連結與畫面截圖	16
5. 工作分配表	17

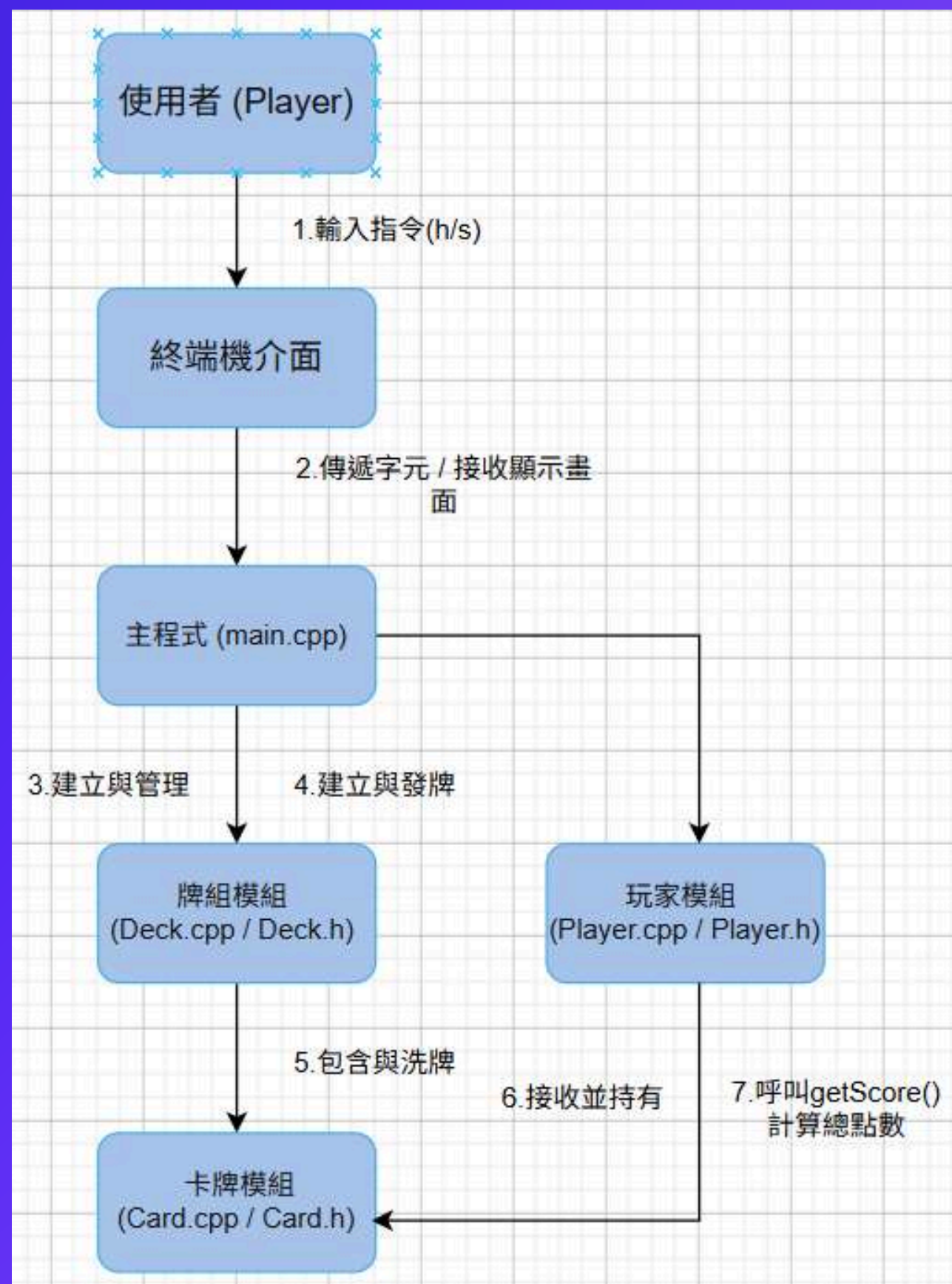


系統之整體介紹

程式模擬撲克牌21點的遊戲規則， 在程式碼中包含了洗牌、發牌、計算點數和輸贏判斷機制， 我們透過物件導向的架構設計， 將現實中的卡牌、牌堆、 玩家與遊戲流程抽象化為程式中的「類別 (Class)」。

系統流程圖

負責遊戲迴圈與總控制



定義花色、數值與點數大小



系統功能描述

- 牌局管理功能：自動產生52張撲克牌，並具有隨機洗牌與發牌功能，確保每張牌在同一局中只會出現一次。
- 動態點數計算：能夠精準計算手牌總和。特別針對撲克牌中的A實作了動態判定功能，會根據當前總和，自動將A視為1點或11點，以最有利於持牌者的方式去計算。
- 玩家互動介面：透過終端機接收玩家指令，執行「要牌」與「停牌」。
- 莊家決策機制：內建自動化莊家邏輯，依21點規則進行，當莊家點數未滿17點必須要牌。
- 狀態與結果顯示：即時更新並顯示玩家及莊家手牌狀態，並在局末準確判定輸贏。

程式架構介紹

```
class Card {
```

```
private:
    string suit; // 花色
    string rank; // 數字 (A,2~10,J,Q,K)

public:
    Card(string s, string r); // 建構子
    int getValue() const;     // 回傳點數
    string toString() const;  // 轉成文字
};
```

Card



類別(定義卡牌)

封裝(private隱藏資料)

將卡牌的「花色」與「數字」設為私有屬性。



封裝(public公開資料)

允許外部「建立卡牌」、「查詢卡牌點數」以及「取得卡牌的文字描述」

Card

```
Card::Card(string s, string r) : suit(s), rank(r) {}

int Card::getValue() const {
    if (rank == "A") return 11;
    if (rank == "J" || rank == "Q" || rank == "K") return 10;
    return stoi(rank);
}

string Card::toString() const {
    return rank + " of " + suit;
}
```

← 建構子初始化變數

← 核心邏輯

(點數換算)

特殊牌：如果是 "A"，預設回傳 11 點。

人頭牌：J、Q、K 一律視為 10 點。

數字牌："2"~"10"，利用 stoi() 函數，把字串直接轉換成對應的整數數字回傳。

← 核心邏輯 (呈現格式)

將內部的資料，組合成易讀的格式。

例如：A of Spades (黑桃A)

Deck

```
class Deck {
```

```
private:
    vector<Card> cards;

public:
    Deck();           // 建立52張牌
    void shuffle();   // 洗牌
    Card deal();      // 發牌
};
```



類別(定義排組)



封裝(private隱藏資料)

將52張卡牌的集合隱藏在內部，保護
資料不被外部直接竄改。

封裝(public公開資料)

允許外部「建構」、「洗牌」、「發牌」

Deck

```
Deck::Deck() {  
    string suits[] = {"Hearts", "Diamonds", "Clubs", "Spades"};  
    string ranks[] = {"2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"};  
  
    for (string s : suits) {  
        for (string r : ranks) {  
            cards.push_back(Card(s, r));  
        }  
    }  
}  
  
void Deck::shuffle() {  
    srand(time(0));  
    random_shuffle(cards.begin(), cards.end());  
}  
  
Card Deck::deal() {  
    Card c = cards.back();  
    cards.pop_back();  
    return c;  
}
```

← 建構子初始化區域變數

核心邏輯 (產生 52 張牌)

利用雙重迴圈，將 4 種花色交錯配對 13 種數字。

每次配對完成就「實例化」一個 Card 物件並塞入牌組中

核心邏輯 (洗牌)

將 vector 內的卡牌順序隨機打亂

核心邏輯 (發牌)

複製牌組最尾端的一張牌，

將那張牌從牌組中剔除，模擬真實發牌

回傳被抽出的牌給呼叫端

Player

```
class Player {
```



類別(定義玩家)

```
private:
    vector<Card> hand; // 手牌

public:
    void addCard(Card c); // 拿牌
    int getScore() const; // 算分數
    void showHand() const; // 顯示手牌
};
```



封装(private隱藏資料)

將玩家目前持有的「手牌」保護起來，外部無法直接塞牌或換牌，

封装(public公開資料)

允許外部「拿牌」、「計算目前分數」、「秀出所有手牌」

Player

```
void Player::addCard(Card c) {
    hand.push_back(c);
}

int Player::getScore() const {
    int total = 0;
    int aces = 0;

    for (auto c : hand) {
        int val = c.getValue();
        if (val == 11) aces++;
        total += val;
    }

    // A 可以變1
    while (total > 21 && aces > 0) {
        total -= 10;
        aces--;
    }

    return total;
}

void Player::showHand() const {
    for (auto c : hand) {
        cout << c.toString() << endl;
    }
    cout << "Score: " << getScore() << endl;
}
```

核心邏輯 (拿牌)

收到一張卡牌後，將它放入手牌陣列中。

核心邏輯 (基本算分與記號)

查看每一張手牌取得點數

如果拿到 A (預設為11點)，就把 A 的數量 +1

將點數先全部加總

核心邏輯 (爆牌救援機制 - A)

如果目前總分超過 21 點，且手邊還有 A，就把一張 A 的點數從 11 變成 1，這個迴圈會執行到總分在 21 以內，或者 A 用光為止

核心邏輯 (顯示狀態)

呼叫 Card 類別的 toString() 印出花色和數字，
最後印出自己的總分。

main

```
#include <iostream>
#include "Deck.h"
#include "Player.h"
#include "Card.h"
using namespace std;

int main() {
    Deck deck;
    Player player;
    Player dealer;

    deck.shuffle(); 呼叫牌組的洗牌功能

    // 初始牌
    player.addCard(deck.deal());
    player.addCard(deck.deal());

    dealer.addCard(deck.deal());
    dealer.addCard(deck.deal());

    cout << "Your hand:\n";
    player.showHand();

    // 玩家回合
    while (player.getScore() < 21) {
        cout << "Hit or Stand (h/s): ";
        char choice;
        cin >> choice;

        if (choice == 'h') {
            player.addCard(deck.deal());
            player.showHand();
        } else break;
    }
}
```

【實例化】 建立一個「牌組」物件 (內部自動包含52張牌)

【實例化】 建立一個「玩家」物件

【實例化】 建立一個「莊家」物件 (共用 Player 類別的邏輯)

只要玩家還沒爆牌(<21)，就能繼續動作

核心邏輯 (玩家要牌)

從牌組抽一張牌給玩家

顯示最新手牌與分數

核心邏輯 (玩家停牌)

選擇 's'，跳出迴圈結束

main

```
// 莊家回合
cout << "\nDealer's turn...\n";
while (dealer.getScore() < 17) {
    dealer.addCard(deck.deal());
}

cout << "Dealer hand:\n";
dealer.showHand();

// 判斷輸贏
int p = player.getScore();
int d = dealer.getScore();

if (p > 21 || (d <= 21 && d > p)) cout << "You lose\n";
else if (p == d) cout << "Tie\n";
else cout << "You win\n";

return 0;
```

核心邏輯 (莊家規則)



根據 21 點規則，莊家點數小於 17 點時
「必須強迫要牌」。



莊家要牌結束後，攤牌顯示最終結果



取得玩家最終分數

取得莊家最終分數

系統使用方式

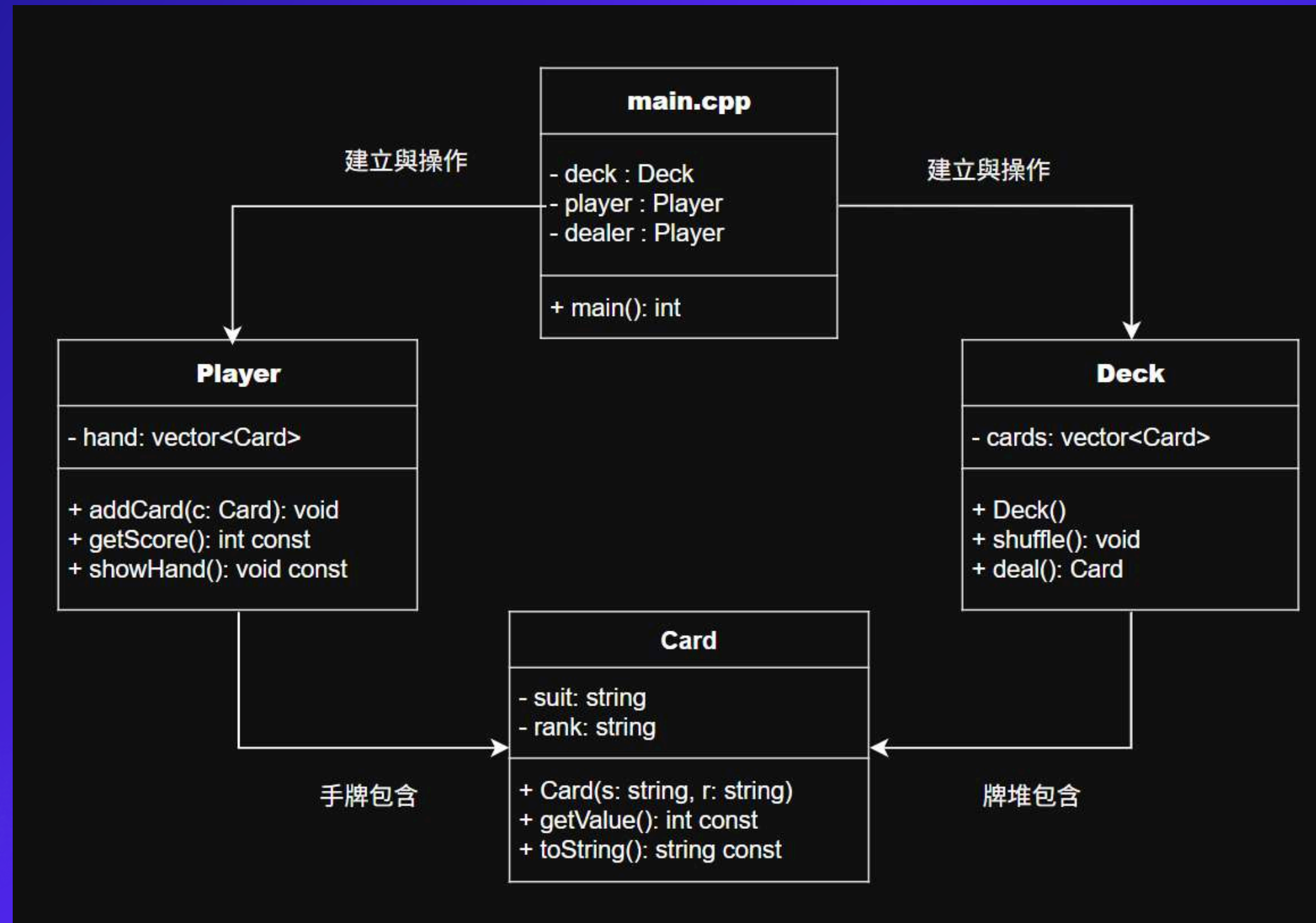
- 開始遊戲： 編譯並執行主程式後， 系統自動發放初始手牌。
- 查看手牌： 畫面會顯示您的兩張明牌與目前總點數， 同時顯示莊家的一張明牌與一張暗牌。
- 進行決策： 系統會提示您輸入選項： 請輸入 H 代表要牌， 或輸入 S 代表停牌。 若輸入 H， 系統會發給您一張新牌並更新畫面。 您可以不斷要牌直到您滿意或不小心爆牌為止。 若輸入 S， 您的回合結束。
- 觀看結果： 您停牌後， 系統會自動翻開莊家暗牌， 並快速展現莊家的補牌過程。 最後畫面會印出雙方最終點數， 以及「YOU WIN!」、「YOU LOSE!」或「TIE!」的對戰結果。

系統運行畫面截圖

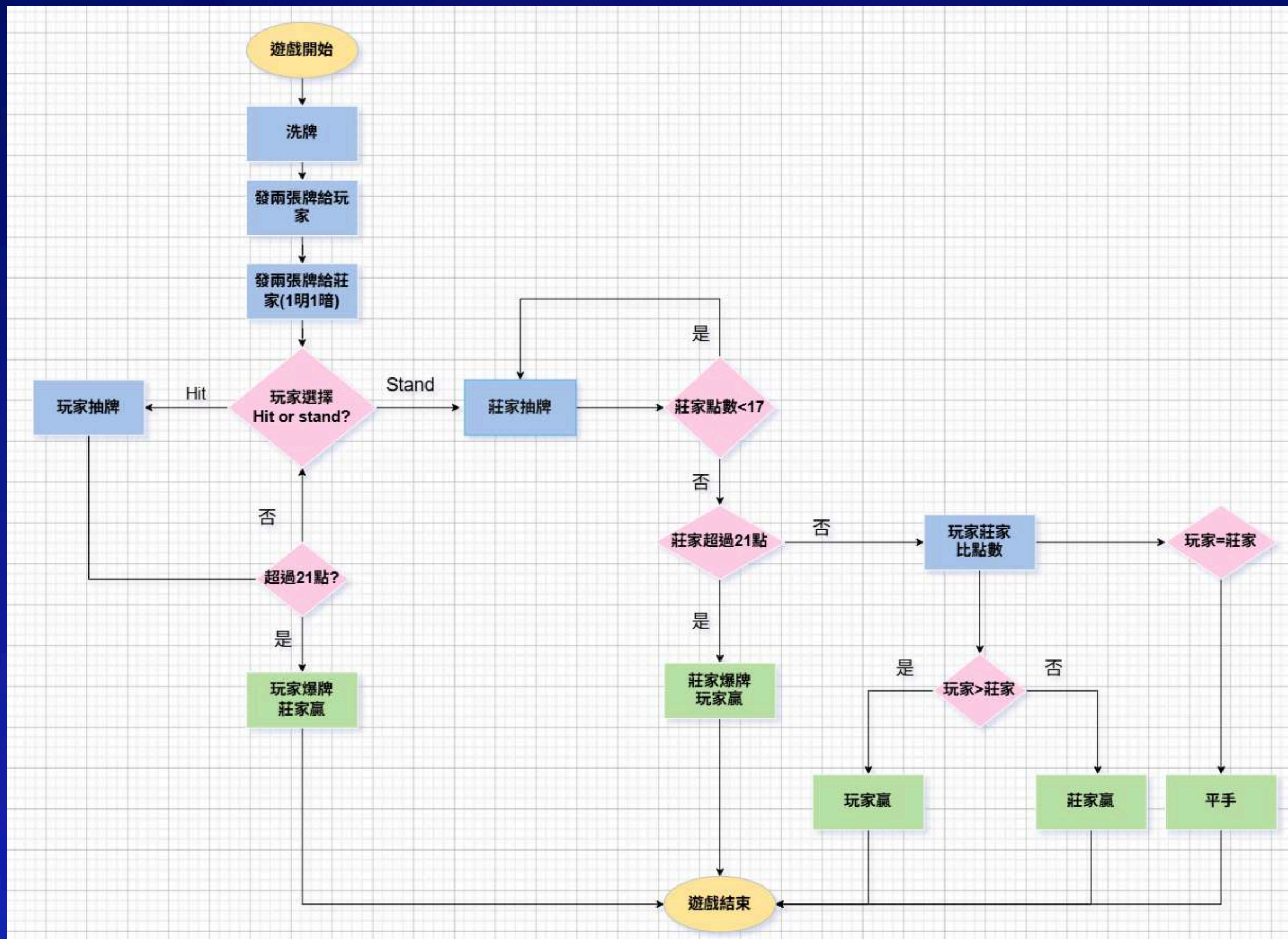
```
Developer PowerShell
+ Developer PowerShell v
*****
** Visual Studio 2026 Developer PowerShell v18.6.2
** Copyright (c) 2026 Microsoft Corporation
*****
PS C:\21點> .\game.exe
Your hand:
3 of Spades
J of Diamonds
Score: 13
Hit or Stand (h/s): h
3 of Spades
J of Diamonds
10 of Spades
Score: 23

Dealer's turn...
Dealer hand:
J of Clubs
K of Clubs
Score: 20
You lose
PS C:\21點>
```

UML圖

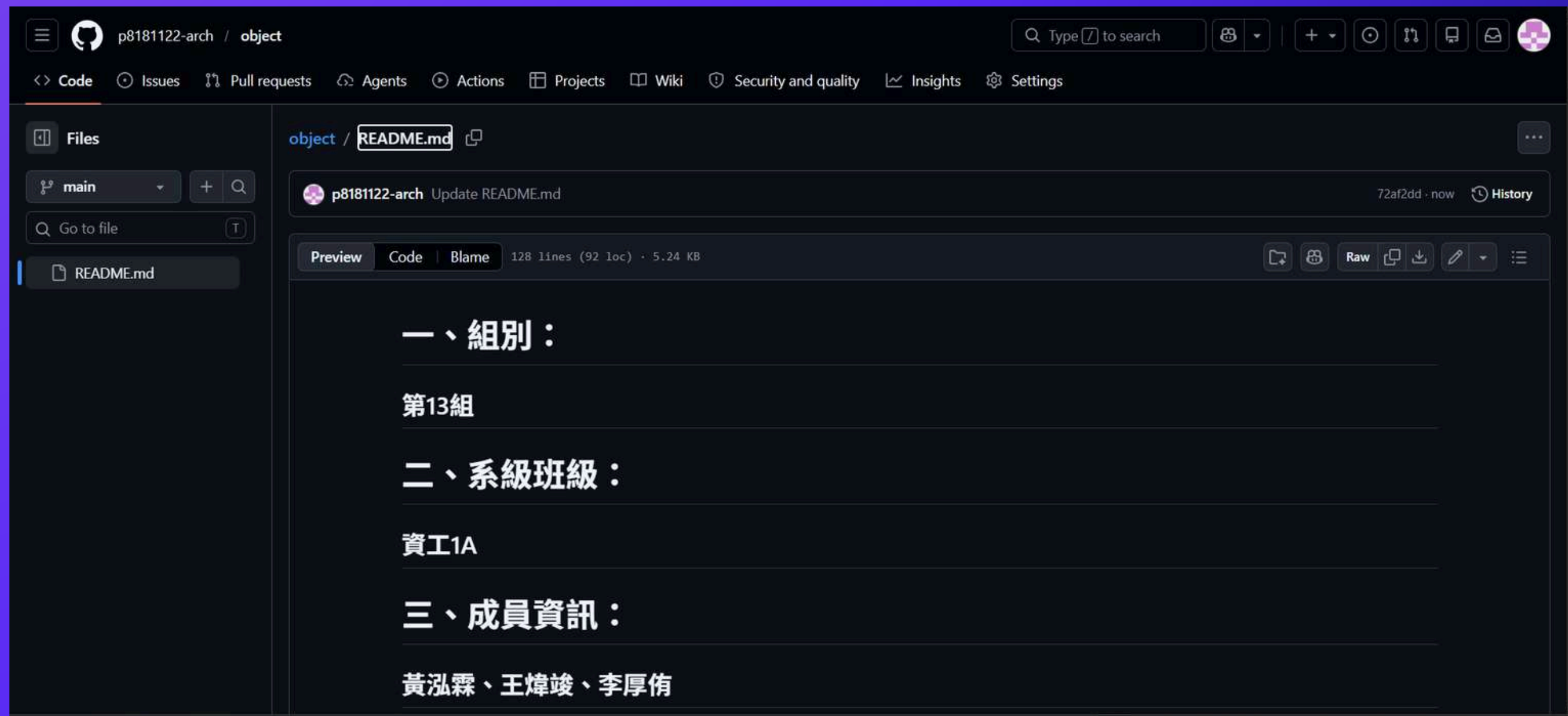


程式運作流程圖

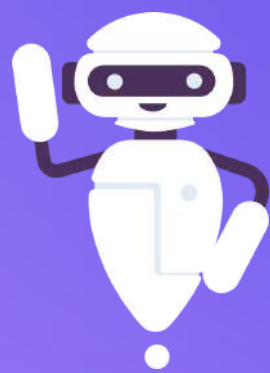


GitHub 連結與畫面截圖

<https://github.com/p8181122-arch/object/edit/main/README.md>

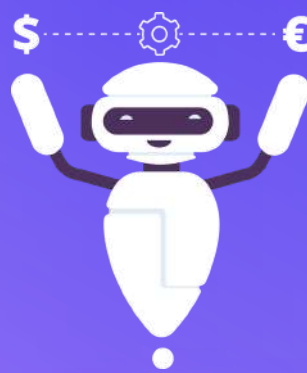


工作分配表：



黃泓霖

- GitHub
- PPT報告
- 上台報告
- 寫程式
- 製作流程圖
- 製作UML圖



王煒竣

- PPT報告
- 上台報告
- 寫程式



李厚侑

- PPT報告
- 上台報告
- 寫程式

THANK YOU!

